- *Multifactor authentication (MFA)*—MFA adds an extra layer of security by prompting users for a secondary method like a one-time password using authenticator apps. This is the most secure method to authenticate your users, but it requires some additional setup to implement.

Which of these options you commit to depends on your use case and how much is at stake. If your web application is small and uses local LLMs without storing any sensitive information, then either the basic username/password or social login options are reasonable starting points. The choice often comes down to ease of implementation versus user convenience.

However, if you are building a platform that stores more than just simple emails or you are accepting payments, then MFA with a robust third-party authentication provider is mandatory. The added complexity of MFA significantly reduces the risk of account compromise by offloading most of the compliance requirements to a third party in exchange for commercial support.

In our case, I present a recommended approach using an external authorization provider that is robust and free for development. Since we are using Next.js, we'll use the NextAuth.js package, which provides a flexible and secure way to implement authentication in Next.js applications. It allows you to easily integrate authentication logic into your serverless functions or API routes.

As for the external authorization provider, we have several great options:

- Clerk.js
- Google Firebase Authentication
- Auth0
- AWS Cognito

Here we focus on Clerk.js. Clerk offers a comprehensive user management and authentication solution that streamlines user onboarding, provides built-in MFA, and handles complex authentication flows. Its generous free tier makes it an excellent choice for development and experimentation.

9.3.1 Simple authentication with Clerk.js and Next.js

The following are the key steps required to configure authentication with Clerk and Next.js:

1. Ensure all users are authenticated before they can use the web app. Upon landing on the index path, they will be redirected to the sign-in page.
2. Redirect authenticated users to the `/chat` endpoint, which is protected.
3. Limit authenticated users to a maximum of 10 messages per day using Upstash Redis.

NOTE This tutorial presents one approach to implementing authentication and authorization using Clerk.js in a Next.js application. There are many ways